MacraScript: A Domain-Specific Language for the Generation of Macramé Patterns

Juan Manuel Serrano Rodriguez - 20211020091, Javier Alejandro Sánchez Salamanca - 20201020167
Department of Systems Engineering
Universidad Francisco José de Caldas
Bogotá, Colombia
{jumserranor, jasanchezs}@udistrital.edu.co

***Abstract*—This paper presents MacraScript, a domain-specific language (DSL) designed and implemented for the creation and visualization of macramé patterns. Macramé, a textile art form based on knotting, has traditionally lacked specialized digital tools for pattern design and verification. MacraScript addresses this gap by providing a declarative syntax that allows users to define patterns programmatically. The language supports two main types of patterns: "Alpha" patterns, which are grid-based and similar to pixel art, and "Normal" patterns, which are based on staggered rows of knots. The compiler, implemented in Python, performs lexical, syntactic, and semantic analysis to generate two distinct graphical outputs: a pixel-art preview (squares for Alpha, diamonds for Normal) and a detailed knot-flow diagram, providing a comprehensive guide for the artisan. This work details the language's design, its final grammar, the compiler architecture, and the resulting visualizations, demonstrating a successful bridge between computational thinking and traditional crafts.**

***Index Terms*—domain-specific language, macramé, compiler, pattern generation, computer graphics, digital crafts**

I. Introduction

Macramé is an ancient textile art form based on creating decorative patterns through organized sequences of knots [3]. Despite its enduring popularity, the design of complex macramé patterns faces significant challenges, including a high dependency on manual planning, a propensity for errors, and a lack of specialized digital tools.

Current methods rely on grid paper, generic graphic design software, or pixel art applications, none of which are optimized for the specific constraints of knot-based textiles. This work introduces MacraScript, a DSL developed to overcome these limitations. MacraScript enables designers to define macramé patterns through code, specifying parameters such as thread count, colors, and knot sequences. The language's compiler generates two key visual outputs: a preview of the final design and a step-by-step knotting guide, effectively bridging the gap between digital design and physical creation.

II. Language Design and Grammar

MacraScript was designed as a declarative, external DSL with a simple and intuitive syntax. The core concept is the `PATTERN`, which can be of type `ALPHA` or `NORMAL`.

Project developed for the Computer Science 3 course.

A. Grammar

The final grammar of MacraScript is defined as follows:

```
<program> ::= "START" <type> <config> <pattern> "END"
<type> ::= "ALPHA" | "NORMAL"
<config> ::= <threads> [<dims>] <colors>
<threads> ::= "THREADS" ":" <INT>
<dims> ::= ("WIDTH" | "HEIGHT") ":" <INT>
<colors> ::= "COLORS" ":" "(" <str_list> ")"
<pattern> ::= "PATTERN" "{" <row_list> "}"
<row_list> ::= <row>+
<row> ::= "ROW" ":" "(" <sequence> ")"
<sequence> ::= <alpha_seq> | <normal_seq>
<alpha_seq> ::= <INT> ("," <INT>)*
<normal_seq> ::= <DIR> ("," <DIR>)*
<DIR> ::= "LEFT" | "RIGHT"
```

Listing 1. Final EBNF Grammar for MacraScript

B. Alpha Patterns

Alpha patterns are defined by a grid of color indices, ideal for creating images. The user specifies the dimensions and the color index for each position in the grid.

```
START ALPHA
THREADS: 4
WIDTH: 4
HEIGHT: 4
COLORS: ("red", "blue")
PATTERN {
    ROW: (0, 1, 0, 1)
    ROW: (1, 0, 1, 0)
    ROW: (0, 1, 0, 1)
    ROW: (1, 0, 1, 0)
}
END
```

Listing 2. Example of an Alpha Pattern

C. Normal Patterns

Normal patterns are defined by rows of knot directions. The language abstracts the complexity of assigning knot pairs; the compiler automatically determines the staggered knotting sequence based on the row index.

```
START NORMAL
THREADS: 8
COLORS: ("red", "blue", "yellow", "green")
PATTERN {
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
    ROW: (LEFT, LEFT, LEFT)
```

```
    ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
}
END
```

Listing 3. Example of a Normal Pattern

III. Compiler Architecture and Implementation

The MacraScript compiler is implemented in Python and follows a traditional three-phase architecture, orchestrated by a main GUI module.

1) **Lexical Analysis:** The `lexical.py` module uses regular expressions to convert the source code into a stream of tokens (e.g., `KEYWORD`, `INTEGER`, `LPAREN`).
2) **Syntactic Analysis:** The `sintactic.py` module employs a simple recursive descent parser to validate the token stream against the language's grammar, building a dictionary that represents the parsed structure.
3) **Semantic Analysis:** The `semantic.py` module interprets the parsed structure. It validates semantic rules and, crucially, translates the high-level row-based definition of `NORMAL` patterns into a concrete list of knot instructions with their corresponding thread pairs.

IV. Results: Visual Outputs

The compiler generates two distinct visualizations for each pattern, displayed in a Tkinter-based GUI.

A. Pixel Art Visualization

This top-view visualization provides a preview of the finished pattern.

- For **Alpha** patterns, it renders a grid of colored squares.
- For **Normal** patterns, it renders a grid of colored diamonds, where each diamond corresponds to a knot. The color of the diamond is determined by the "active" thread in the knot (the one that ties the knot), ensuring consistency with the knot-flow diagram.

B. Knot-Flow Diagram

This visualization serves as a step-by-step guide for the artisan.

- For **Alpha** patterns, it shows a snake-like flow of nodes, where each node's color and connecting line correspond to the pattern's color grid.
- For **Normal** patterns, it displays a detailed lattice diagram. It shows how threads cross over and under each other, with circles representing the knots. The color of each circle represents the active thread, and an arrow indicates the knot's direction (left or right).

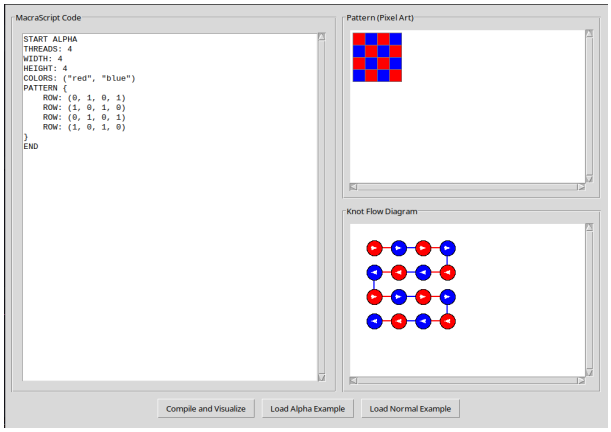

Fig. 1. Final dual visualization for an Alpha pattern.

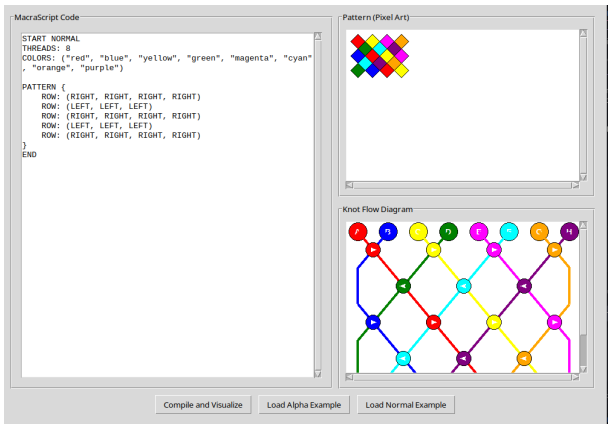

Fig. 2. Final dual visualization for a Normal pattern.

V. Conclusion

MacraScript successfully demonstrates the application of DSL principles to a traditional craft. By abstracting the complexities of macramé pattern design into a simple, declarative language, it lowers the barrier to entry for creating complex designs and reduces the potential for manual error. The implementation of a full compiler that generates dual, coherent visual outputs provides a powerful tool for macramé artisans. Future work could involve expanding the library of knots, developing more advanced visualization techniques, and building a community platform for sharing patterns.

References

[1] Fowler, M. (2010). Domain-Specific Languages. Addison-Wesley Professional.

[2] Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. ACM Computing Surveys (CSUR), 37(4), 316-344.

[3] Karner, C. (2005). Macramé: The craft of knotting. Schiffer Publishing.